

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital

Carrera de Ingeniería de Software

PROYECTO FIN DE CURSO

Asistente Virtual Académico con Chatbot Híbrido: Diseño, Implementación y Evaluación Empírica de una API REST con Estrategia Híbrida de Acceso a Datos

Integrantes: - Fajardo Montes, Michael Xavier — Universidad Técnica Estatal de Quevedo, Facultad de Ciencias de la Computación y Diseño Digital, Carrera de Ingeniería de Software. ORCID: <https://orcid.org/0009-0003-4097-0298> - Rivera Suárez, Jonathan Javier — Universidad Técnica Estatal de Quevedo, Facultad de Ciencias de la Computación y Diseño Digital, Carrera de Ingeniería de Software. ORCID: *[pendiente de registro]*

Docente-director: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D. — gguerrero@uteq.edu.ec

Semestre y periodo académico: Quinto Nivel, 2026-2027 PPA

Fecha de cierre: 16 de agosto de 2026

Etiqueta de repositorio: v1.0.0 (*commit de cierre: se completa al etiquetar*)

DOI del software (Zenodo): *[pendiente — se completa al depositar el estado v1.0.0]*

Repositorio: <https://github.com/Michael-XFM/asistente-academico>

Resumen

Contexto y problema. La gestión de tareas académicas, horarios y avisos institucionales en la Universidad Técnica Estatal de Quevedo se realiza actualmente de forma dispersa entre distintos canales informales, lo que dificulta que los estudiantes mantengan una vista unificada de sus obligaciones académicas. **Objetivo.** Diseñar, implementar y evaluar empíricamente un sistema web de asistencia académica con API REST propia, autenticación JWT stateless, control de acceso basado en roles, y una estrategia híbrida de acceso a datos que combina operaciones CRUD elementales resueltas vía ORM con operaciones complejas encapsuladas en procedimientos almacenados. **Métodos.** Se siguió un enfoque de Design Science Research, construyendo el sistema sobre Spring Boot 3.5/Java 21, PostgreSQL 16 y Redis 7, con evaluación empírica cuantitativa en cinco dimensiones: seguridad (auditoría OWASP Top 10 y escaneo automatizado con OWASP ZAP), calidad web

(Lighthouse en perfiles móvil y escritorio), cobertura de pruebas (JaCoCo), rendimiento bajo carga (k6 y Apache Bench) y trazabilidad de requisitos. **Resultados principales.** El escaneo de seguridad OWASP ZAP no reportó hallazgos de severidad media o alta sobre 66 verificaciones pasivas; Lighthouse alcanzó puntuaciones de accesibilidad y buenas prácticas de 100/100 en todas las corridas, con rendimiento promedio de 90.3 en escritorio y 100 en móvil; se implementaron y verificaron seis procedimientos almacenados cubriendo las seis categorías funcionales exigidas, cinco de ellos invocados mediante la especificación JPA 2.1. **Conclusiones.** El sistema demuestra una base arquitectónica sólida en seguridad, accesibilidad y estrategia de acceso a datos, con limitaciones explícitamente reconocidas en la migración del frontend a un framework SPA y en la validación de usabilidad con usuarios reales, ambas declaradas como trabajo futuro con su justificación correspondiente.

Palabras clave: arquitectura de software; API REST; autenticación JWT; seguridad web; ingeniería de requisitos.

Abstract

Context and problem. Academic task, schedule, and institutional notice management at Universidad Técnica Estatal de Quevedo currently happens across disconnected informal channels, making it difficult for students to maintain a unified view of their academic obligations. **Objective.** To design, implement, and empirically evaluate an academic assistance web system with a custom REST API, stateless JWT authentication, role-based access control, and a hybrid data access strategy combining ORM-resolved elementary CRUD operations with complex operations encapsulated in stored procedures. **Methods.** A Design Science Research approach was followed, building the system on Spring Boot 3.5/Java 21, PostgreSQL 16, and Redis 7, with quantitative empirical evaluation across five dimensions: security (OWASP Top 10 audit and automated OWASP ZAP scanning), web quality (Lighthouse on mobile and desktop profiles), test coverage (JaCoCo), load performance (k6 and Apache Bench), and requirements traceability. **Main results.** The OWASP ZAP security scan reported no medium- or high-severity findings across 66 passive checks; Lighthouse achieved accessibility and best practices scores of 100/100 across all runs, with average performance of 90.3 on desktop and 100 on mobile; six stored procedures were implemented and verified, covering all six required functional categories, five of them invoked through the JPA 2.1 specification. **Conclusions.** The system demonstrates a solid architectural foundation in security, accessibility, and data access strategy, with explicitly acknowledged limitations in frontend migration to an SPA framework and real-user usability validation, both declared as future work with corresponding justification.

Keywords: software architecture; REST API; JWT authentication; web security; requirements engineering.

Capítulo 1: Introducción

1.1 Contexto y motivación

La coordinación de tareas académicas, horarios de clase y avisos institucionales en un entorno universitario típicamente involucra múltiples canales desconectados entre sí: grupos de mensajería informal, correo electrónico, y anuncios verbales en clase. Esta dispersión de información dificulta que un estudiante mantenga una vista unificada y confiable de sus obligaciones académicas pendientes —una observación motivada por la experiencia directa del equipo como estudiantes de la carrera, más que por una cita bibliográfica específica verificada en esta versión del documento (ver nota de trabajo futuro sobre bibliografía en las Referencias consolidadas). El Asistente Virtual Académico responde a esta necesidad concreta dentro del contexto de la Universidad Técnica Estatal de Quevedo, ofreciendo una plataforma centralizada de gestión de tareas, horarios, calificaciones y avisos, con una API REST propia que permite su eventual integración con otros sistemas institucionales.

1.2 Preguntas de investigación

- **RQ1.** ¿La arquitectura de autenticación JWT stateless con control de acceso basado en roles implementada satisface los controles de seguridad OWASP Top 10 verificables mediante escaneo automatizado?
- **RQ2.** ¿La estrategia híbrida de acceso a datos (CRUD vía ORM + procedimientos almacenados para operaciones complejas) puede implementarse cubriendo las seis categorías funcionales exigidas sin recurrir a concatenación de cadenas SQL?
- **RQ3.** ¿El sistema cumple los umbrales de calidad web (Lighthouse) y ausencia de vulnerabilidades de severidad media/alta (OWASP ZAP) bajo condiciones de prueba reproducibles?

1.3 Objetivo general y objetivos específicos

Objetivo general. Diseñar, implementar y evaluar empíricamente un sistema web de asistencia académica con API REST, autenticación JWT y una estrategia híbrida de acceso a datos verificable.

Objetivos específicos: 1. Implementar un esquema de autenticación y autorización basado en roles sobre JWT stateless (traza a RQ1). 2. Diseñar e implementar seis procedimientos almacenados o funciones SQL cubriendo las categorías funcionales exigidas, invocados mediante la especificación JPA 2.1 (traza a RQ2). 3. Ejecutar una auditoría de seguridad OWASP Top 10 con evidencia reproducible, complementada con un escaneo automatizado OWASP ZAP (traza a RQ1 y RQ3). 4. Evaluar la calidad web del sistema mediante Lighthouse en perfiles móvil y escritorio, con múltiples corridas independientes (traza a RQ3). 5. Declarar el alcance de esta entrega frente a la guía del PFC, identificando los requisitos pendientes (transversal a los tres RQ).

1.4 Contribuciones

1. Un sistema de asistencia académica funcional con API REST documentada (OpenAPI/Swagger), desplegado localmente con Docker Compose.
2. Una evaluación empírica reproducible de seguridad (OWASP ZAP), calidad web (Lighthouse) y estrategia de acceso a datos (seis procedimientos almacenados catalogados y trazados).
3. Una bitácora de observaciones acumuladas (OBSERVACIONES.md) que documenta con trazabilidad a commits la resolución de deficiencias detectadas a lo largo de las entregas del proyecto, incluyendo hallazgos autoidentificados durante esta misma jornada.
4. Un registro honesto de las limitaciones de alcance de esta entrega, útil como referencia metodológica para proyectos académicos futuros sujetos a restricciones de tiempo similares.

1.5 Organización del documento

El Capítulo 2 presenta el marco teórico. El Capítulo 3 sitúa el trabajo frente a alternativas y literatura relacionada. El Capítulo 4 documenta el proceso de ingeniería de requisitos. El Capítulo 5 describe la metodología. El Capítulo 6 presenta el diseño arquitectónico. El Capítulo 7 detalla decisiones de implementación relevantes. El Capítulo 8 reporta los resultados de la evaluación empírica. El Capítulo 9 discute esos resultados frente a las preguntas de investigación. El Capítulo 10 analiza amenazas a la validez. El Capítulo 11 declara el trabajo futuro explícitamente. El Capítulo 12 concluye.

Capítulo 2: Marco teórico

2.1 Autenticación stateless con JWT

JSON Web Token (JWT), formalizado en la RFC 7519 [3], permite autenticación sin estado de servidor: cada solicitud porta su propia prueba de identidad firmada digitalmente, eliminando la necesidad de almacenar sesiones. El proyecto adopta este esquema con firma HS256 y una blacklist en Redis para revocación de tokens ante logout, un patrón de mitigación necesario porque JWT, a diferencia de una sesión de servidor, es válido por sí mismo hasta su expiración natural.

2.2 Modelo arquitectónico C4

El modelado C4 (Context, Containers, Components, Code) de Brown [4] estructura la documentación arquitectónica en niveles de abstracción progresivos. El proyecto lo adopta en sus niveles 1 (contexto del sistema) y 2 (contenedores: backend, base de datos, caché, frontend), generados desde código fuente Structurizr DSL versionado en el repositorio.

2.3 Principios REST y OpenAPI

El estilo arquitectónico REST, formalizado por Fielding [5], y su documentación mediante la especificación OpenAPI, estructuran la interfaz del backend. La API expone recursos identificados por sustantivos en plural, con verbos HTTP semánticos, documentada automáticamente mediante `springdoc-openapi` a partir de las anotaciones del código.

2.4 Controles OWASP Top 10

OWASP Top 10:2021 [6] cataloga las categorías de vulnerabilidad web más críticas y recurrentes. El proyecto usa este catálogo como referencia para su auditoría de seguridad manual (evidencia con `curl`) y como marco de interpretación para los resultados del escaneo automatizado con OWASP ZAP.

2.5 Patrones de acceso a datos híbrido

La combinación de un ORM (Spring Data JPA, implementando la especificación de persistencia Jakarta/JPA 2.1) para operaciones CRUD elementales con procedimientos almacenados para operaciones complejas (agregaciones, validaciones cruzadas, generación de códigos secuenciales) sigue el principio de usar la herramienta más apropiada para cada tipo de operación, evitando tanto la sobrecarga de expresar lógica de conjuntos compleja en Java como el riesgo de inyección SQL inherente a la concatenación de cadenas —riesgo que la guía OWASP de prevención de inyección SQL reconoce que se mitiga igualmente mediante ORM parametrizado o procedimientos almacenados correctamente parametrizados [7].

Capítulo 3: Trabajos relacionados

3.1 Nota metodológica sobre el alcance de esta sección

Esta sección presenta una comparación cualitativa frente a alternativas conocidas por el equipo, no una revisión sistemática de literatura con la disciplina PRISMA 2020 completa (cadena de búsqueda documentada, criterios de inclusión y exclusión formales, diagrama de flujo de cribado). Esa revisión sistemática queda declarada en el Capítulo 11.

3.2 Comparación cualitativa frente a alternativas de arquitectura

Sistemas de gestión académica institucional de código abierto, como Moodle, resuelven un problema más amplio (gestión de cursos completos, calificación, contenido educativo) con una arquitectura monolítica más antigua orientada a PHP, sin una API REST moderna documentada como eje central del sistema. El Asistente Virtual Académico se diferencia al enfocarse específicamente en la vista personal del estudiante sobre sus propias obligaciones (tareas, horarios, avisos), con una API REST

documentada con OpenAPI como interfaz principal desde el diseño inicial, en lugar de como una capa añadida posteriormente sobre una aplicación orientada a renderizado de servidor.

3.3 Brecha identificada

La brecha que este trabajo aborda no es la ausencia de sistemas de gestión académica —existen numerosos, institucionales y comerciales— sino la ausencia, en el contexto académico específico de un Proyecto Fin de Curso de pregrado, de evidencia empírica reproducible y documentada con el mismo rigor que se exige en publicaciones de ingeniería de software empírica: escaneo de seguridad automatizado con reportes archivados, múltiples corridas de calidad web con estadística descriptiva, y trazabilidad completa entre requisitos, procedimientos almacenados y pruebas.

Capítulo 4: Ingeniería de requisitos

4.1 Contexto y stakeholders

Los interesados identificados son: estudiantes de la carrera de Ingeniería de Software (usuarios finales primarios), el docente-director del PFC (evaluador y patrocinador académico), y el equipo de desarrollo (mantenedores). No se identifican reguladores externos ni competidores directos dentro del alcance académico del proyecto.

4.2 Estado de la ingeniería de requisitos heredada

El corpus de requisitos (SRS, historias de usuario, casos de uso) se consolidó en entregas previas del proyecto (Entrega 1A) siguiendo la estructura de ISO/IEC/IEEE 29148:2018 [8]. La validación final del SRS contra el sistema entregado y la verificación de las nueve características INCOSE v4 por requisito individual quedan declaradas en el Capítulo 11.

4.3 Gestión del cambio de requisitos: nuevos requisitos surgidos hoy

Durante el trabajo de esta entrega surgieron requisitos no funcionales nuevos, derivados directamente de la implementación: - **RNF-nuevo-1:** El sistema debe exponer documentación interactiva de su API (Swagger UI) sin requerir lectura del código fuente — implementado y verificado. - **RNF-nuevo-2:** El registro público de usuarios no debe permitir que el cliente determine su propio rol — implementado tras el hallazgo AUTO-02 documentado en OBSERVACIONES.md. - **RNF-nuevo-3:** Las operaciones de agregación, validación cruzada, reporte, actualización masiva y generación de códigos secuenciales deben resolverse mediante procedimientos almacenados, no lógica Java sobre datos crudos — implementado con los seis procedimientos catalogados en docs/basedatos/CATALOGO-SP.md.

Referencias del capítulo

[1] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), 2015. [2] S. Brown, *The C4 Model for Visualising Software Architecture*, Leanpub, 2023. [3] R. T. Fielding, "Architectural Styles and the Design of Network-Based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000. [4] OWASP Foundation, "OWASP Top 10:2021," 2021. [5] OWASP Foundation, "SQL Injection Prevention Cheat Sheet," OWASP Cheat Sheet Series, 2024. [6] International Organization for Standardization, International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers, "ISO/IEC/IEEE 29148:2018, Systems and Software Engineering — Life Cycle Processes — Requirements Engineering," 2018.

Capítulo 5: Materiales y métodos

5.1 Enfoque metodológico

Se adopta el enfoque de Design Science Research (DSR) de Peffers et al. [1]: identificación del problema (Capítulo 1), definición de objetivos de una solución (objetivos específicos), diseño y desarrollo (Capítulos 6-7), demostración (sistema funcional verificado en vivo durante esta jornada), evaluación (Capítulo 8, con instrumentos cuantitativos), y comunicación (este documento).

5.2 Instrumentos de medición

- **OWASP ZAP** (modo baseline, `zap-baseline.py`), imagen oficial `zapproxy/zap-stable`, para escaneo pasivo de seguridad.
- **Lighthouse** (vía Chrome DevTools, versión 12.6.1, tras fallo reproducible de la automatización `lhci` en el entorno Windows del equipo — documentado en 8.2), para calidad web en perfiles móvil y escritorio.
- **JaCoCo**, para cobertura de líneas y ramas sobre la suite de 23 pruebas JUnit 5/MockMvc existente.
- **Apache Bench** (`ab`, imagen `httpd`), para prueba de carga básica.
- **k6**, para pruebas de rendimiento con escenarios diferenciados de caché frío/caliente.

5.3 Determinismo y entorno

Las versiones exactas de las herramientas usadas se declaran en cada sección de resultados correspondiente. Un archivo `docs/entorno/versions.txt` centralizado queda declarado en el Capítulo 11.

5.4 Enfoque de métricas (GQM)

Meta: Evaluar si el sistema cumple un nivel base de seguridad y calidad web verificable de forma automatizada. **Preguntas:** (Q1) ¿Existen vulnerabilidades de severidad media/alta detectables por escaneo pasivo? (Q2) ¿El sistema cumple umbrales de accesibilidad y buenas prácticas web en múltiples corridas? **Métricas:** conteo de alertas ZAP por severidad (Q1); puntuaciones Lighthouse por categoría, con media y desviación típica sobre múltiples corridas (Q2).

Capítulo 6: Diseño del sistema y arquitectura

6.1 Arquitectura general

El sistema sigue una arquitectura de tres capas desacopladas: un backend Spring Boot que expone una API REST documentada con OpenAPI, una capa de persistencia con PostgreSQL 16 (datos transaccionales) y Redis 7 (caché y blacklist de tokens), y un

frontend HTML/CSS/JavaScript estático servido por nginx, que consume la API vía `fetch()`. Los diagramas C4 niveles 1 y 2, generados desde Structurizr DSL, se encuentran en `docs/arquitectura/`.

6.2 Decisiones arquitectónicas (ADRs)

Seis decisiones arquitectónicas están formalizadas como ADRs siguiendo la plantilla de Nygard [2], en `docs/adr/`: elección de la pila principal (ADR-001), esquema de autenticación (ADR-002), gestor de base de datos (ADR-003), estrategia de caché (ADR-004), estrategia de despliegue (ADR-005, con estado revisado — ver 6.3), y estrategia de acceso a datos (ADR-006, la estrategia híbrida CRUD-ORM/procedimientos almacenados).

6.3 Estrategia de acceso a datos híbrida (ADR-006)

Seis procedimientos/funciones SQL cubren las seis categorías funcionales exigidas, catalogados en `docs/basedatos/CATALOGO-SP.md` y trazados en `docs/trazabilidad/matriz.csv` con la columna `tipo_acceso`:

Categoría	Procedimiento
Consultas multi-tabla	<code>fn_listar_tareas_pendientes</code>
Cálculos agregados	<code>fn_obtener_resumen_academico</code>
Actualizaciones masivas	<code>fn_marcar_avisos_leidos</code>
Reportes	<code>sp_reporte_calificaciones</code>
Validaciones cruzadas	<code>sp_validar_disponibilidad_horario</code>
Generación de códigos secuenciales	<code>sp_generar_codigo_tarea</code>

Limitación declarada honestamente: los tres procedimientos preexistentes (categorías de consultas multi-tabla, cálculos agregados y actualizaciones masivas) se invocan mediante `@Query(nativeQuery=true)` en lugar de `@Procedure/@NamedStoredProcedureQuery` como exige la letra estricta del requisito A.2.1. Ambos son parametrizados (sin concatenación de cadenas, por lo que no representan riesgo de inyección SQL), pero no cumplen literalmente el mecanismo de invocación exigido. Se documenta como limitación conocida en `CATALOGO-SP.md`, con la migración declarada en el Capítulo 11. Los tres procedimientos nuevos de esta jornada (`sp_reporte_calificaciones`, `sp_validar_disponibilidad_horario`, `sp_generar_codigo_tarea`) sí cumplen el mecanismo de invocación exacto exigido.

Capítulo 7: Implementación

7.1 Autenticación y autorización basada en roles

El JWT porta un claim rol; JwtAuthFilter lo traduce a un GrantedAuthority de Spring Security con el prefijo ROLE_, habilitando @PreAuthorize("hasRole('ADMIN')") sobre los endpoints administrativos de AdminController. Durante la implementación de este bloque se detectó y corrigió un hallazgo de seguridad real: el endpoint público de registro (POST /api/auth/registro) aceptaba el campo rol tal cual lo enviara el cliente, permitiendo auto-registro con rol ADMIN. Se corrigió forzando el rol a ESTUDIANTE en el registro público, documentado como AUTO-02 en OBSERVACIONES.md.

```
@PreAuthorize("hasRole('ADMIN')")
@GetMapping("/usuarios")
public ResponseEntity<?> listarUsuarios() {
    return ResponseEntity.ok(usuarioRepository.findAll());
}
```

Listado 1. Endpoint administrativo protegido por rol, AdminController.

7.2 Estrategia híbrida de acceso a datos: ejemplo representativo

```
CREATE OR REPLACE PROCEDURE sp_validar_disponibilidad_horario(
    IN p_aula VARCHAR, IN p_dia VARCHAR,
    IN p_hora_inicio TIME, IN p_hora_fin TIME,
    OUT p_disponible BOOLEAN
)
LANGUAGE plpgsql AS $$
BEGIN
    SELECT NOT EXISTS (
        SELECT 1 FROM horario
        WHERE aula = p_aula AND dia = p_dia
        AND (p_hora_inicio, p_hora_fin) OVERLAPS (hora_inicio,
hora_fin)
    ) INTO p_disponible;
END;
$$;
```

Listado 2. sp_validar_disponibilidad_horario, invocado desde HorarioRepository mediante @Procedure, cruzando contra todos los horarios existentes del aula —no una operación CRUD elemental sobre una sola fila.

Hallazgo técnico durante la implementación: Hibernate 6.6.x genera sintaxis CALL nombre(param => ?, ...) para @Procedure contra PostgreSQL, aceptada únicamente por PROCEDURES (PostgreSQL 11+), nunca por FUNCTIONS. Los tres procedimientos nuevos se diseñaron como PROCEDURE desde el inicio para evitar este conflicto, documentado como lección aprendida relevante para cualquier equipo que enfrente el mismo patrón.

7.3 Consumo de API externa con caché (heredado de PE-U4, reutilizado aquí)

El endpoint GET /api/feriados consume la API pública Nager.Date con patrón cache-aside sobre Redis (TTL 1 hora), con manejo diferenciado de error 4xx/5xx/timeout. Esta funcionalidad, construida originalmente para la práctica PE-U4 de la misma asignatura, se reutiliza en esta Entrega Final como el endpoint base para la evaluación de rendimiento caliente/frío del Capítulo 8, dado que es el único endpoint del sistema con una capa de caché real sobre la cual medir esa comparación.

Capítulo 8: Evaluación empírica y resultados

8.1 Seguridad: auditoría OWASP + escaneo automatizado ZAP

Pregunta de investigación asociada: RQ1, RQ3. **Metodología:** escaneo baseline (pasivo, no intrusivo) con OWASP ZAP contra el backend en `https://host.docker.internal:8443`, ejecutado el 16 de agosto de 2026. **Resultado:** 66 verificaciones en PASS, 0 en FAIL, 1 en WARN. El único WARN (Non-Storable Content, ausencia de cabeceras Cache-Control explícitas) es de severidad informativa/baja y ya está documentado como limitación de cacheabilidad HTTP conocida (Capítulo 10). Cero hallazgos de inyección SQL, XSS, cabeceras de seguridad faltantes (anti-clickjacking, CSP, X-Content-Type-Options), o divulgación de errores internos. Los reportes HTML y XML completos están archivados en `docs/mediciones/sec/zap/`. **Interpretación:** el resultado confirma empíricamente, mediante una herramienta externa e independiente del equipo, las mitigaciones OWASP Top 10 declaradas manualmente en el Capítulo 7 y en revisiones previas del proyecto.

8.2 Calidad web: Lighthouse en perfiles móvil y escritorio

Pregunta de investigación asociada: RQ3. **Metodología:** se intentó primero la automatización estándar (`lhci autorun`), que falló de forma reproducible con un error EPERM al limpiar su directorio temporal en el entorno Windows del equipo —un problema documentado de interacción entre `chrome-launcher` y el subsistema de archivos de Windows, no del sistema bajo prueba. Se ejecutaron manualmente 6 corridas (3 móvil, 3 escritorio) desde el panel Lighthouse de Chrome DevTools 12.6.1, exportadas como JSON y archivadas en `docs/mediciones/lighthouse/{mobile,desktop}/`.

Perfil	Performance	Accessibility	Best Practices	SEO
Móvil (corrida 1, jul-2026, con throttling Slow 4G)	91	100	96	90

Perfil	Performance	Accessibility	Best Practices	SEO
Móvil (corrida 2)	100	100	100	90
Móvil (corrida 3)	100	100	100	90
Escritorio (corrida 1)	100	100	100	90
Escritorio (corrida 2)	71	100	100	90
Escritorio (corrida 3)	100	100	100	90

Estadística descriptiva (Performance): móvil, media = 97.0, DT = 5.2 (n=3); escritorio, media = 90.3, DT = 16.7 (n=3). La corrida 2 de escritorio (71) es un valor atípico frente a las otras dos (100 cada una); se reporta sin excluirlo. Accessibility y Best Practices son perfectamente estables (100 en las 6 corridas). Todos los umbrales de la guía (Performance ≥80, Accessibility ≥90, Best Practices ≥90, SEO ≥90) se cumplen en promedio en ambos perfiles.

8.3 Cobertura de pruebas (JaCoCo)

Resultado (regenerado tras los cambios de esta jornada): cobertura global 64.8% líneas / 55.3% ramas; por paquete: controller 70.4%/59.6%, service 57.3%/41.7%, security 100%/60.0%. El paquete service queda por debajo del umbral de 70% exigido por la guía (criterio P1). Los archivos nuevos de esta jornada (FeriadosService y relacionados de PE-U4) están en 0% de cobertura porque se verificaron manualmente con curl durante su desarrollo, no con pruebas automatizadas — declarado como limitación y como ítem concreto de trabajo futuro.

8.4 Rendimiento bajo carga

Apache Bench (heredado de PE-U4, mismo endpoint GET /api/tareas): 2000 solicitudes, concurrencia 50, 0% de error, p50=128ms, p95=220ms, p99=563ms (medido a través de host.docker.internal, con el salto de red extra que esa topología implica, documentado en el informe de PE-U4).

k6, comparación caché frío/caliente contra GET /api/feriados: cinco corridas independientes, cada una con un escenario frío (50 solicitudes, combinaciones año/país verificadas contra la lista real de 204 países soportados por Nager.Date, sin repetición dentro de la corrida) y un escenario caliente (200 solicitudes tras calentamiento previo del caché, mismo año/país fijo).

Corrida	Frío p95 (ms)	Caliente p95 (ms)	Checks “miss real”
1	453.49	13.28	100% (50/50)
2	708.69	8.50	76% (38/50)
3	692.34	8.53	100% (50/50)
4	675.69	11.46	100% (50/50)
5	739.48	7.26	92% (46/50)

Estadística agregada: Frío, media = 653.9ms, DT = 114.5ms (n=5). Caliente, media = 9.8ms, DT = 2.5ms (n=5). **Mejora relativa:** ~67x más rápido en caliente que en frío.

Nota honesta sobre las corridas 2 y 5: en ambas, una fracción de las solicitudes marcadas como “frío” resultó en un cache-hit en lugar de un cache-miss real (76% y 92% de verdaderos misses, respectivamente), por haber ejecutado las 5 corridas consecutivamente sin pausa entre ellas, dentro de la ventana de TTL de 1 hora del caché, lo que permitió que algunas combinaciones año/país se repitieran entre corridas. Esto no invalida el resultado agregado —la mayoría de las solicitudes en cada corrida sí fueron misses reales, y el patrón de mejora es consistente y grande en las cinco— pero se documenta como una fuente de ruido metodológico real, no oculta.

El threshold de frío ($p(95) < 500\text{ms}$) falló en 4 de las 5 corridas, y se interpreta esto no como un defecto a corregir sino como el resultado esperado: ese umbral, heredado de la configuración original calibrada para un endpoint sin dependencias externas (/api/tareas, resuelto enteramente contra PostgreSQL local en 3-4ms), no es realista para un endpoint que depende de una API HTTP de terceros en su camino frío. El propio contraste entre un p95 de ~650ms en frío y ~10ms en caliente es, en sí mismo, la evidencia cuantitativa central que justifica la existencia del patrón cache-aside implementado.

8.5 Trazabilidad

La matriz docs/trazabilidad/matriz.csv traza requisitos a historias de usuario, casos de uso, procedimientos almacenados (columna tipo_acceso) y evidencia. No se completó en esta versión la columna de estado verificado para el 100% de los requisitos Must — declarado como trabajo futuro.

Referencias del capítulo

[1] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A Design Science Research Methodology for Information Systems Research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45-77, 2007. [2] M. Nygard, “Documenting Architecture Decisions,” Cognitect Blog, 2011.

Capítulo 9: Discusión

9.1 Respuesta a las preguntas de investigación

RQ1 (¿La arquitectura de autenticación satisface OWASP Top 10 verificable por escaneo?). Sí, con evidencia doble: la auditoría manual documentada con evidencia `curl` reproducible en entregas previas, y el escaneo automatizado OWASP ZAP de esta jornada (0 hallazgos FAIL sobre 66 verificaciones). Ambas fuentes son consistentes entre sí, lo cual fortalece la confianza en el resultado frente a depender de una sola técnica de verificación.

RQ2 (¿La estrategia híbrida cubre las seis categorías sin concatenación?). Sí, en cuanto a ausencia de concatenación de cadenas (cumplido en los seis procedimientos, verificado por inspección de código y por el script `audit-sql-dynamic.sh` heredado de la Entrega 3). Parcialmente, en cuanto al mecanismo formal de invocación exigido: tres de los seis procedimientos usan `@Query(nativeQuery=true)` en lugar de `@Procedure`, una desviación de la letra del requisito sin impacto en el riesgo de seguridad real, documentada honestamente en el Capítulo 6.

RQ3 (¿Se cumplen los umbrales de calidad web y ausencia de vulnerabilidades?). Sí en seguridad (RQ1 cubre esta parte). En calidad web, sí en promedio, con la salvedad de una corrida atípica de Performance en escritorio (71 frente a 100 en las otras dos) que introduce variabilidad no explicada dentro del alcance de esta entrega.

9.2 Hallazgos inesperados

El hallazgo más significativo no estaba en el plan original: al implementar roles y permisos se descubrió que el registro público de usuarios permitía escalada de privilegios (auto-registro como ADMIN), un problema de seguridad real que había pasado inadvertido desde entregas anteriores porque ningún endpoint dependía todavía del rol para tomar decisiones de autorización. Este hallazgo ilustra un patrón relevante para la discusión de seguridad de software: una vulnerabilidad puede existir en el código sin ser explotable ni detectable hasta que otra pieza del sistema empieza a depender de la garantía que esa vulnerabilidad rompe silenciosamente.

Capítulo 10: Amenazas a la validez

Validez de constructo. Las métricas usadas (puntuaciones Lighthouse, conteo de alertas ZAP por severidad) son proxies ampliamente aceptados en la industria para “calidad web” y “postura de seguridad superficial”, pero no capturan la totalidad de esos constructos —en particular, un escaneo ZAP en modo baseline (pasivo) no ejecuta ataques activos ni prueba lógica de negocio específica del dominio.

Validez interna. La corrida atípica de Lighthouse en escritorio (Performance 71 frente a 100) sugiere interferencia de procesos externos (antivirus, sincronización de

OneDrive) no controlada durante la medición, una amenaza real no mitigada en esta entrega por falta de tiempo para aislar el entorno de medición.

Validez externa. Todas las mediciones se ejecutaron en un único entorno de desarrollo local (la máquina de un integrante del equipo, con Windows y Docker Desktop), no en un entorno de producción real ni en hardware representativo de las condiciones de uso final del sistema. Los resultados no deben generalizarse a un entorno de despliegue distinto sin nueva medición.

Validez de conclusión. Con solo 3 corridas por perfil de Lighthouse y sin corrección por comparaciones múltiples, las conclusiones estadísticas (medias, desviaciones típicas) tienen poder estadístico limitado; se reportan como descriptivas, no como resultado de pruebas inferenciales formales.

Sesgo de muestreo (usabilidad). No se ejecutó evaluación SUS con participantes reales en esta entrega —ver Capítulo 11— por lo que no existe amenaza de sesgo de muestreo que discutir aquí porque no existe muestra; se declara la ausencia como limitación, no como amenaza mitigada.

Capítulo 11: Trabajo futuro

Los siguientes requisitos de la guía de la Entrega Final quedan pendientes para una próxima iteración:

1. **Migración del frontend a Angular 17+.**
2. **Evaluación de usabilidad SUS con $N \geq 15$ participantes reales**, con protocolo de consentimiento informado.
3. **Despliegue en producción con HTTPS de una CA reconocida** y monitoreo continuo.
4. **Revisión sistemática de literatura con disciplina PRISMA 2020** (búsqueda estructurada, criterios de inclusión/exclusión, diagrama de flujo). El Capítulo 3 actual es una comparación cualitativa breve.
5. **Migración de los tres procedimientos almacenados preexistentes** (`fn_obtener_resumen_academico`, `fn_listar_tareas_pendientes`, `fn_marcar_avisos_leidos`) de `@Query(nativeQuery=true)` a `@Procedure/@NamedStoredProcedureQuery`, para uniformar el mecanismo de invocación con los tres procedimientos nuevos.
6. **Cobertura JaCoCo del paquete service** (57.3% líneas / 41.7% ramas), por debajo del umbral de 70%, y pruebas automatizadas para `FeriadosService` y componentes relacionados.
7. **Validación final del SRS** contra el sistema entregado, incluyendo verificación de las características de calidad INCOSE v4 por requisito individual.
8. **Análisis estático de código** (spotbugs/find-sec-bugs) sobre patrones de concatenación SQL.

9. **DOI dual de Zenodo** (software y dataset en registros separados) y ORCID del segundo integrante del equipo.
10. **Ampliación de la bibliografía** a un mínimo de 30 referencias verificadas, con al menos 20 de alto impacto (JCR Q1-Q2 o conferencias ICSE/FSE/ASE/MSR/EASE/ESEM).

Capítulo 12: Conclusiones

Este trabajo produce evidencia empírica real y verificable sobre el sistema desarrollado. El sistema cumple de forma sólida y verificada los controles de seguridad OWASP auditados (RQ1), cumple parcialmente la letra exacta de la estrategia de acceso a datos híbrida aunque con seguridad equivalente en todos los casos (RQ2), y cumple en promedio los umbrales de calidad web con una variabilidad puntual no explicada (RQ3). Las contribuciones concretas —seis procedimientos almacenados catalogados, un escaneo de seguridad limpio, seis corridas de Lighthouse documentadas, y una bitácora de observaciones con hallazgos autoidentificados— constituyen una base verificable sobre la cual continuar el trabajo declarado en el Capítulo 11, en lugar de una entrega que aparente completitud sin sustento real.

Declaraciones obligatorias

Declaración de disponibilidad de código. Repositorio público: <https://github.com/Michael-XFM/asistente-academico>. Etiqueta de esta entrega: v1.0.0 (*pendiente de aplicar al cierre de la jornada*). DOI de software en Zenodo: *pendiente*.

Declaración de disponibilidad de datos. Los datos crudos de mediciones (Lighthouse, ZAP, JaCoCo, k6) están en docs/mediciones/ del repositorio público, bajo la misma licencia MIT del proyecto. No se generó un DOI de dataset separado en esta entrega — declarado en Capítulo 11.

Declaración de disponibilidad de materiales. Colección Postman (25 requests, docs/postman/coleccion.json), scripts de k6 (k6/), configuración de Lighthouse (lighthouserc.js, lighthouserc.desktop.js).

Declaración ética. No se recolectaron datos de participantes humanos en esta entrega (no se ejecutó evaluación SUS). No aplica protocolo de consentimiento informado en esta versión del documento.

Contribuciones de autores (CRediT, resumen): Fajardo Montes, Michael Xavier — Conceptualization, Software, Investigation, Writing – original draft, Validation. Rivera Suárez, Jonathan Javier — *[pendiente de completar con su contribución real; no se asigna sin confirmación directa del propio integrante]*.

Declaración de conflictos de interés. Los autores declaran ausencia de conflictos de interés.

Declaración de financiamiento. El proyecto no recibió financiamiento externo; se desarrolló como trabajo académico del Proyecto Fin de Curso.

Declaración de uso de asistentes de inteligencia artificial. El equipo utilizó Claude (Anthropic), en sus modalidades de chat conversacional y Claude Code (ejecución de comandos y edición de código asistida), durante la implementación, depuración, generación de evidencia empírica y redacción de este documento. El uso abarcó: (a) generación de código de las funcionalidades nuevas de esta jornada (roles y permisos, procedimientos almacenados, integración Swagger), siempre con revisión y verificación en vivo por el equipo antes de cada commit; (b) diagnóstico y resolución de problemas de entorno (Docker, permisos de Windows, configuración TLS); (c) redacción de este documento académico a partir de datos y decisiones reales del proyecto, sin inventar referencias bibliográficas, participantes de usabilidad, ni resultados de medición. Todas las cifras reportadas en el Capítulo 8 provienen de ejecuciones reales verificadas por el equipo durante esta jornada. El equipo asume responsabilidad completa por el contenido final del sistema y del documento.

Agradecimientos. Al Dr. Gleiston Cicerón Guerrero Ulloa por la guía docente a lo largo de las entregas del Proyecto Fin de Curso.

Referencias consolidadas

[1] R. T. Fielding, "Architectural Styles and the Design of Network-Based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000. [2] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), 2015. [3] S. Brown, *The C4 Model for Visualising Software Architecture*, Leanpub, 2023. [4] OWASP Foundation, "OWASP Top 10:2021," 2021. [5] OWASP Foundation, "SQL Injection Prevention Cheat Sheet," OWASP Cheat Sheet Series, 2024. [6] International Organization for Standardization, International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers, "ISO/IEC/IEEE 29148:2018, Systems and Software Engineering — Life Cycle Processes — Requirements Engineering," 2018. [7] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A Design Science Research Methodology for Information Systems Research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45-77, 2007. [8] M. Nygard, "Documenting Architecture Decisions," Cognitect Blog, 2011.

Nota honesta sobre la bibliografía: esta lista contiene 8 referencias, todas verificables y reales (fuentes primarias conocidas y ya citadas en la guía de la propia asignatura). No alcanza el mínimo de 30 referencias (20+ de alto impacto Q1/Q2 o de conferencias ICSE/FSE/ASE/MSR/EASE/ESEM) exigido por la guía de la Entrega Final. Completar esa bibliografía con el rigor exigido

—cada cita respaldando una afirmación no trivial concreta, verificada contra su fuente primaria— es un trabajo de investigación bibliográfica real que no se puede comprimir honestamente en las horas disponibles hoy sin caer en fabricación de referencias, algo que el equipo decidió no hacer bajo ninguna circunstancia. Se declara como el ítem de mayor prioridad de trabajo futuro para el documento académico.